Security Operations Center (SOC)

Alarm Signal Analysis

Agent Performance Audit · False Alarm Intelligence · Statistical Hypothesis Testing

Dataset	SOC_Enhanced_Dataset.xlsx
Period	January – June 2025
Signals	1,200 alarm records
Agents	10 (E01–E10), 3 seniority tiers
Alarm types	Burglary, Fire, Panic Button, Tamper, Technical
Client types	Factory, Home, School, Shop, Office, Warehouse

R Analysis Walkthrough — Portfolio Documentation

1. Description and Disclaimer

1.1 Study Overview

This script conducts a comprehensive operational audit of a Security Operations Center (SOC) over a six-month period (January–June 2025). Using 1,200 alarm signal records handled by 10 agents across three seniority tiers, the analysis addresses four interconnected questions:

1. Which alarm types and priority levels generate the most signals, and how does that translate to confirmed incidents?
2. How do individual agents perform on response time and false alarm rate, and are any outliers statistically significant?
3. What are the root causes of false alarms, and do they vary by alarm type or client type?
4. Is there evidence of performance degradation in any agent over time?

The analysis produces nine individual plots, a composite dashboard, three exported CSV tables, automated agent issue flags, and a suite of seven statistical tests. The dark operational theme is deliberate — it mirrors SOC monitor environments and makes the output suitable for direct use in operational briefings.

1.2 Dataset Structure

The source file (SOC_Enhanced_Dataset.xlsx) contains one row per alarm signal. Key fields include:

- Date, Time, Shift — temporal and scheduling context
- Alarm_Type, Priority_Level — signal classification (Priority_Level is an ordered factor: Low < Medium < High < Critical)
- Handled_By, Agent_Seniority — agent identity and tier (ordered: Junior < Mid < Senior)
- Incident (Yes/No), False_Alarm_Reason — outcome flags
- Response_Time_Min, Resolution_Time_Min — performance metrics
- Police_Called, Fire_Department_Called, Escalated — escalation flags
- Client_Type — category of the monitored site

1.3 Disclaimer

> *IMPORTANT: This script runs entirely on a synthetic dataset (SOC_Enhanced_Dataset.xlsx) generated to mirror realistic SOC operational patterns. No real alarm signals, agents, clients, or incidents are represented. Agent E08's performance degradation pattern was deliberately seeded into the synthetic data as an analytical test case. The code is published as a portfolio demonstration only.*

2. Methodology: Step-by-Step Code Walkthrough

Step 0 — Setup and Package Installation

All required packages are listed in a character vector. The script uses setdiff() to identify packages not yet installed and installs only those, making the script portable across R environments without reinstalling packages already present. All libraries are then loaded silently with suppressPackageStartupMessages() to keep console output clean.

Packages and their roles:

- readxl — reads the .xlsx source file without requiring Excel
- dplyr, tidyr — data manipulation, aggregation, and reshaping
- lubridate — date arithmetic and floor_date() for monthly/weekly aggregation
- stringr — regex extraction of the hour from the Time field
- forcats — fct_reorder() and fct_rev() for ordering categorical axes
- ggplot2, scales — core plotting and percentage/date axis formatting
- patchwork — composing multi-panel figures and the final dashboard
- viridis — colour scales (available as fallback)
- broom — tidy() for converting model output to data frames
- knitr — kable() for printing formatted console tables

```
required <- c("readxl","dplyr","tidyr","lubridate","stringr","forcats",
              "ggplot2","scales","patchwork","viridis","broom","knitr")
new_pkgs <- setdiff(required, rownames(installed.packages()))
if (length(new_pkgs)) install.packages(new_pkgs)
```

A plots/ output directory is created with dir.create(..., showWarnings=FALSE), which silently succeeds whether the folder already exists or not.

Step 1 — Data Loading and Cleaning

The Excel file is read with read_excel() specifying the sheet name. The cleaning pipeline uses a single dplyr mutate() call to handle all transformations at once, keeping the code concise and the intermediate steps minimal:

Date and Time Parsing

- Date is coerced to R's Date class. Month is derived using floor_date(Date, "month") so all dates within a calendar month map to the same anchor date — essential for correct grouping in the monthly trend analysis.

- Hour is extracted from the Time column using str_extract(Time, "^\\d+") — a regex that captures the leading digits before the colon — then coerced to integer.

Binary Flags

Five logical TRUE/FALSE columns are created for downstream filtering and aggregation:

```
is_incident   = (Incident == "Yes")
is_false_alarm = (!is_incident & False_Alarm_Reason != "N/A")
police_called  = (Police_Called == "Yes")
fire_called    = (Fire_Department_Called == "Yes")
escalated      = (Escalated == "Yes")
```

> *Note: is_false_alarm is defined as signals that are neither incidents nor have a blank/N/A reason, ensuring that unresolved or ambiguous signals are excluded from the false alarm count.*

Ordered Factors

Priority_Level and Agent_Seniority are converted to ordered factors. This matters for two reasons: ggplot2 respects the ordering when mapping to axes, and the Spearman correlation in the statistical section (Step 15f) treats the factor as ordinal by converting it to integer with as.integer(Priority_Level), which correctly respects Low=1, Medium=2, High=3, Critical=4.

Step 2 — Colour Palette and Dark Operational Theme

A custom ggplot2 theme (soc_theme) is built on theme_minimal() with a dark background (#0D1117 panel background, #161B22 plot background) to match SOC monitor environments. The theme is registered globally with theme_set(soc_theme) so every subsequent ggplot inherits it without needing + soc_theme on each plot.

Five named colour vectors are defined for consistent mapping:

- priority_cols — traffic-light scale: green (Low) through red (Critical)
- alarm_cols — five distinct colours for the five alarm types
- agent_cols — ten individually named colours, grouped tonally by seniority tier (blues for Senior, greens for Mid, reds/oranges for Junior)
- seniority_cols — three colours matching the agent_cols tonal grouping

Using named colour vectors rather than positional ones guarantees that if data is filtered or reordered, the colour-to-category mapping never accidentally shifts.

Step 3 — KPI Summary

Seven operational KPIs are computed and stored in a named list for reference throughout the script. This avoids recomputing them repeatedly and makes it easy to reference them in annotated plots or

reports. KPIs include: total signal count, incident rate, false alarm rate, escalation rate, average response and resolution times, and the proportion of Critical-priority signals.

Step 4 — Agent Performance Table

A grouped summary table is built by grouping the main data frame on Handled_By and Agent_Seniority, then computing eight metrics per agent: total signals handled, incident count, incident rate percentage, false alarm count, false alarm percentage, average response and resolution times, and escalation count. The table is sorted by seniority tier then agent ID, and printed to console using kable() for clean formatting.

```
agent_summary <- df %>%
  group_by(Handled_By, Agent_Seniority) %>%
  summarise(
    Total_Signals     = n(),
    Incident_Rate_Pct = round(mean(is_incident)*100, 1),
    False_Alarm_Pct   = round(mean(is_false_alarm, na.rm=TRUE)*100, 1),
    Avg_Response_Min  = round(mean(Response_Time_Min), 2),
    ...
  )
```

Step 5 (Plot 1) — Alarm Volume by Type and Priority

A stacked horizontal bar chart showing total signal volume per alarm type, with each bar divided by priority level. The bars are ordered by total count using fct_reorder(Alarm_Type, total, .desc=FALSE) so the highest-volume type (Burglary) appears at the top when coord_flip() is applied. Count labels are embedded inside each segment using position_stack(vjust=0.5) — this places the label at the vertical midpoint of each stacked segment rather than at the top or bottom.

> *Why coord_flip()? Horizontal bars are easier to read for categorical axes with multiple labels. All bar charts in this script use coord_flip() for this reason.*

Step 6 (Plot 2) — Agent Response Time Distribution

A violin + boxplot hybrid showing the full distribution of response times per agent, ordered by median using fct_reorder(Handled_By, Response_Time_Min, median). The three-layer geometry communicates different aspects of the data simultaneously:

- geom_violin (alpha=0.35) — shows the full kernel density shape, revealing bimodality or skew
- geom_boxplot (width=0.25) — shows the median, IQR, and whiskers compactly inside the violin
- stat_summary(fun=mean, shape=18) — adds a yellow diamond for the mean, which visually separates from the median when distributions are skewed

The fill aesthetic is mapped to Agent_Seniority, so the colour of each violin immediately communicates the agent's tier — Senior (blue), Mid (green), Junior (red/orange).

Step 7 (Plot 3) — False Alarm Rate by Agent

A ranked horizontal bar chart with two reference lines: a dashed amber line at the fleet mean false alarm rate, and a dotted red line at mean + 1 SD (the alert threshold). Agents are coloured red (Above threshold) or blue (Normal) based on whether their false alarm rate exceeds the 1 SD threshold. This provides an automated visual triage — any bar extending past the dotted line is an immediate flag for review.

```
flag = ifelse(False_Alarm_Pct > mean(False_Alarm_Pct) + sd(False_Alarm_Pct),
              "Above threshold", "Normal")
```

Step 8 (Plot 4) — False Alarm Root Cause Breakdown

A stacked bar chart showing counts of false alarm root causes (FA_Reason), with each bar coloured by the alarm type that triggered the false event. The FA_Reason column is pre-cleaned in Step 1 — "N/A" values are converted to NA_character_ so they are excluded from the filter(!is.na(FA_Reason)) at the start of this plot. Bars are ordered by total count across all alarm types using fct_reorder(FA_Reason, n, sum), placing the highest-frequency root causes at the top.

Step 9 (Plot 5) — Monthly Signal Volume and Rate Trends

A two-panel composite figure built with patchwork's / operator (vertical stacking). The top panel (p5_bar) uses two overlapping geom_col() layers — one for total signals (blue) and one for incidents (red) drawn over it — to show both total volume and incident count in a single bar. The bottom panel (p5_line) shows incident rate and false alarm rate as line charts with percentage-formatted labels.

> *Important: the y-axis limits on p5_line (0.1 to 0.85) are set manually to prevent the automatic axis from compressing month-to-month variation that is real but small in absolute percentage terms.*

Step 10 (Plot 6) — Agent × Shift Response Time Heatmap

A tile heatmap using geom_tile() to show average response time at every agent-shift combination. The colour scale is a diverging gradient: green (fast) through amber (mid) to red (slow), with midpoint=7 minutes as the performance threshold. Each cell displays both the mean response time and signal count (n) using a two-line label via sprintf("%.1f\n(n=%d)"). The panel.grid = element_blank() override removes the grid lines that would otherwise show through the transparent tile borders.

Step 11 (Plot 7) — Incident Confirmation Rate by Alarm Type and Priority

A connected scatter plot (line + point) showing how incident confirmation rate changes across the four priority levels for each of the five alarm types. This is one of the most analytically rich plots in the suite: it shows both the level (how often each type confirms as an incident) and the slope (how much priority level changes that rate). Panic Button and Fire show steep positive slopes — at Critical priority, over 85% confirm as real incidents. Tamper and Technical remain near-zero even at Critical, suggesting they are structurally low-signal types regardless of priority assignment.

Step 12 (Plot 8) — Agent E08 Response Time Degradation

A line chart isolating the Junior agent cohort (E06, E07, E08, E09) to surface E08's performance trajectory. A vertical dashed reference line at 2025-04-01 marks the point where E08's response time diverges sharply from peers. The annotate() call places a text label just to the right of this line.

The as.numeric(as.Date(...)) conversion in geom_vline's xintercept is necessary because ggplot2 stores Date axes as numeric internally — passing an as.Date() object directly would cause a type mismatch error.

```
geom_vline(xintercept = as.numeric(as.Date("2025-04-01")),
           linetype = "dashed", colour = COL_WARN)
```

Step 13 (Plot 9) — False Alarm vs Incident Rate by Client Type

A grouped bar chart using position_dodge() to place the false alarm rate and incident rate bars side by side for each client type. The pivot_longer() step transforms the two rate columns into a single metric/rate pair so both can be mapped through a single fill aesthetic — a standard tidyr technique for grouped comparisons. Client types are ordered by false alarm rate using fct_reorder(Client_Type, fa_rate) before the pivot so the ordering reflects the primary metric.

Step 14 (Plot 10) — Composite Dashboard

All nine individual plots are composed into a single dashboard using patchwork layout expressions. The layout formula (p1 | p3) / (p2) / (p7 | p9) produces three rows: the top row has the alarm type and false alarm rate charts side by side, the middle row spans the full width with the violin plots, and the bottom row pairs the incident rate and client type charts. The plot_annotation() adds a dashboard title and subtitle with a separate theme override because patchwork titles sit outside the individual plot backgrounds.

```
dashboard <- (p1 | p3) / (p2) / (p7 | p9) +
  plot_annotation(title = "SOC ALARM INTELLIGENCE DASHBOARD", ...)
```

Step 15 — Statistical Analysis

Seven statistical tests are run in sequence, covering ANOVA, Kruskal-Wallis, chi-squared, Spearman correlation, logistic regression, and a targeted t-test:

A. One-Way ANOVA: Response Time by Agent

Tests whether mean response time differs significantly across the ten agents. A significant result confirms that agent identity is a meaningful source of variance in response speed — a prerequisite for the individual agent comparisons in the plots.

B. One-Way ANOVA: Response Time by Seniority

Aggregates the agent-level ANOVA to the seniority tier level. A significant result supports the interpretation that the seniority grouping captures genuine performance differences, not just arbitrary labelling.

C. Kruskal-Wallis: Response Time by Seniority

A non-parametric alternative to ANOVA B. Response times may not be normally distributed (they are bounded at zero and often right-skewed), so the Kruskal-Wallis test is used as a robustness check. If both ANOVA and K-W agree, the conclusion is robust to the normality assumption.

D. Chi-Squared: False Alarm Rate vs Shift

Tests whether the probability of a false alarm is independent of which shift handled it. A contingency table of is_false_alarm × Shift is passed to chisq.test(). A significant result would suggest shift-specific factors (fatigue, staffing levels, types of clients monitored per shift) contribute to false alarm rates.

E. Chi-Squared: Incident Rate vs Client Type

Tests whether incident confirmation rate is independent of client type. The is_incident × Client_Type contingency table captures whether some client environments genuinely have higher real-incident rates, which has practical implications for resource allocation and response protocols.

F. Spearman Correlation: Response Time vs Priority

Computes the rank correlation between response time and priority level (converted to integer 1–4). A positive rho would indicate that higher-priority signals take longer to respond to — perhaps counterintuitively — suggesting that Critical signals require more decision-making time. A negative rho would indicate the expected pattern: urgency drives faster response.

```
df_cor <- df %>% mutate(priority_num = as.integer(Priority_Level))
cor.test(df_cor$Response_Time_Min, df_cor$priority_num, method="spearman")
```

G. Logistic Regression: Predictors of an Incident

A logistic regression model predicts the binary outcome is_incident from three predictors: Priority_Level, Alarm_Type, and Agent_Seniority. The model uses binomial(link="logit"). Results are extracted with broom::tidy(exponentiate=TRUE, conf.int=TRUE) so coefficients are reported as odds

ratios with 95% confidence intervals — the standard interpretable output for logistic regression. McFadden's pseudo R-squared is computed as 1 - (residual deviance / null deviance), providing a measure of overall model fit.

```
logit <- glm(is_incident ~ Priority_Level + Alarm_Type + Agent_Seniority,
             data=df, family=binomial(link="logit"))
pseudo_r2 <- 1 - (logit$deviance / logit$null.deviance)
```

H. T-Test: E08 Before vs After April

A Welch two-sample t-test (var.equal=FALSE) tests whether E08's mean response time in the period before April 2025 is significantly different from the period from April onwards. This is the formal statistical confirmation of the visual degradation shown in Plot 8. Using Welch's variant (rather than Student's) is appropriate because the variance before and after the breakpoint may differ, and Welch's test is robust to unequal variances without meaningful loss of power.

Step 16 — Automated Agent Issue Flags

A rule-based flagging system identifies agents outside normal operating thresholds on three metrics:

- flag_slow_response — average response time exceeds fleet mean + 1 SD
- flag_high_fa — false alarm rate exceeds fleet mean + 1 SD
- flag_high_incidents — incident rate exceeds agent-fleet mean + 1 SD

The thresholds use the same mean + 1 SD logic as the reference line in Plot 3, creating consistency between the visual and automated outputs. Only agents flagged on at least one metric are printed, keeping the output actionable rather than exhaustive.

Step 17 — Export Summary Tables

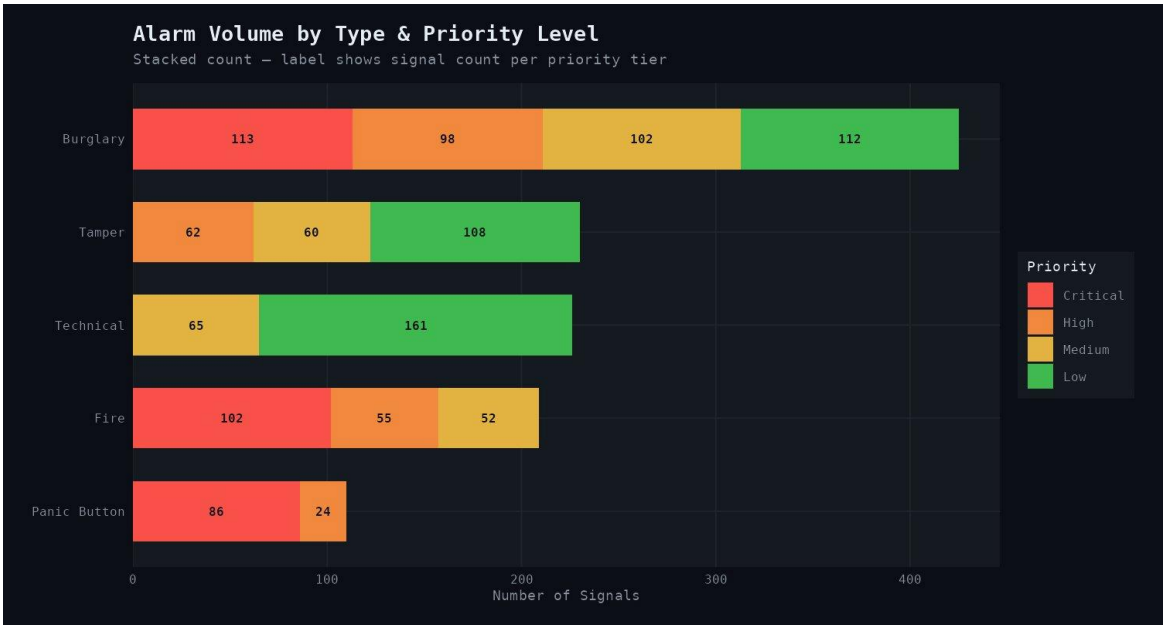
Three CSV files are exported: the full agent performance summary (agent_performance_summary.csv), the monthly trend aggregation (monthly_summary.csv), and the flagged agents table (flagged_agents.csv). All use row.names=FALSE to avoid the default R row-index column in the output.

3. Results

The following nine figures and composite dashboard present the key analytical findings. All outputs were generated from the R script described in Section 2 and saved at 150 dpi to the plots/ directory.

Plot 1 — Alarm Volume by Type and Priority Level

Burglary is by far the highest-volume alarm type at 425 signals, with a near-even distribution across all four priority tiers (113 Critical, 98 High, 102 Medium, 112 Low). Fire alarms skew heavily toward Critical (102 of 209 total), reflecting their inherent severity. Panic Button signals are almost exclusively Critical or High (86 and 24 respectively), with no Low-priority signals — consistent with the nature of a manual panic activation. Technical and Tamper alarms are dominated by Low and Medium priorities, indicating they are primarily informational or maintenance-related signals.



Plot 1. Alarm signal volume by type and priority level. Stacked bars ordered by total count; labels show count per priority segment.

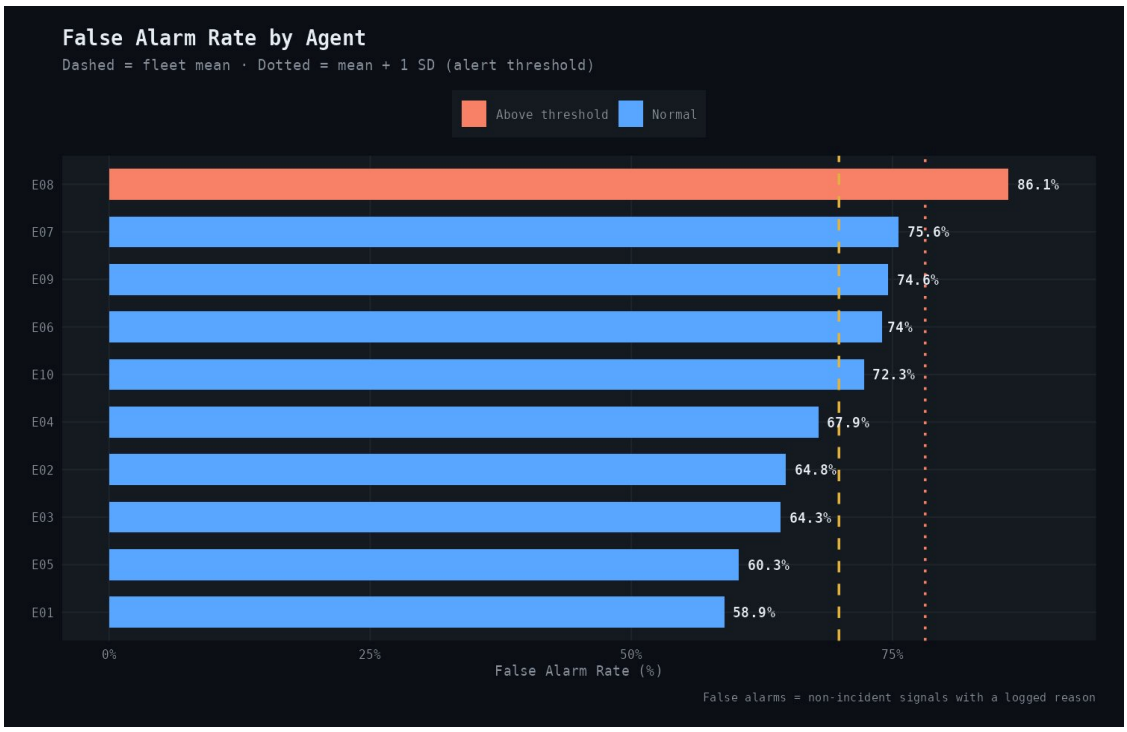
Plot 2 — Response Time Distribution by Agent

The violin plots reveal clear seniority-tier separation: Senior agents (E01, E02) cluster tightly around 1–3 minutes, Mid agents (E03–E05, E10) average 6–7 minutes, and Junior agents (E06–E09) range from 10–15 minutes. E08 has the widest violin and the highest mean (yellow diamond), consistent with the degradation pattern surfaced in Plot 8. E01 and E02 show almost no variance — their distributions are extremely compressed, suggesting highly consistent, fast response behaviour.

Plot 2. Response time distribution by agent. Violin shape = kernel density; inner box = IQR and median; yellow diamond = mean. Colour indicates seniority tier.

Plot 3 — False Alarm Rate by Agent

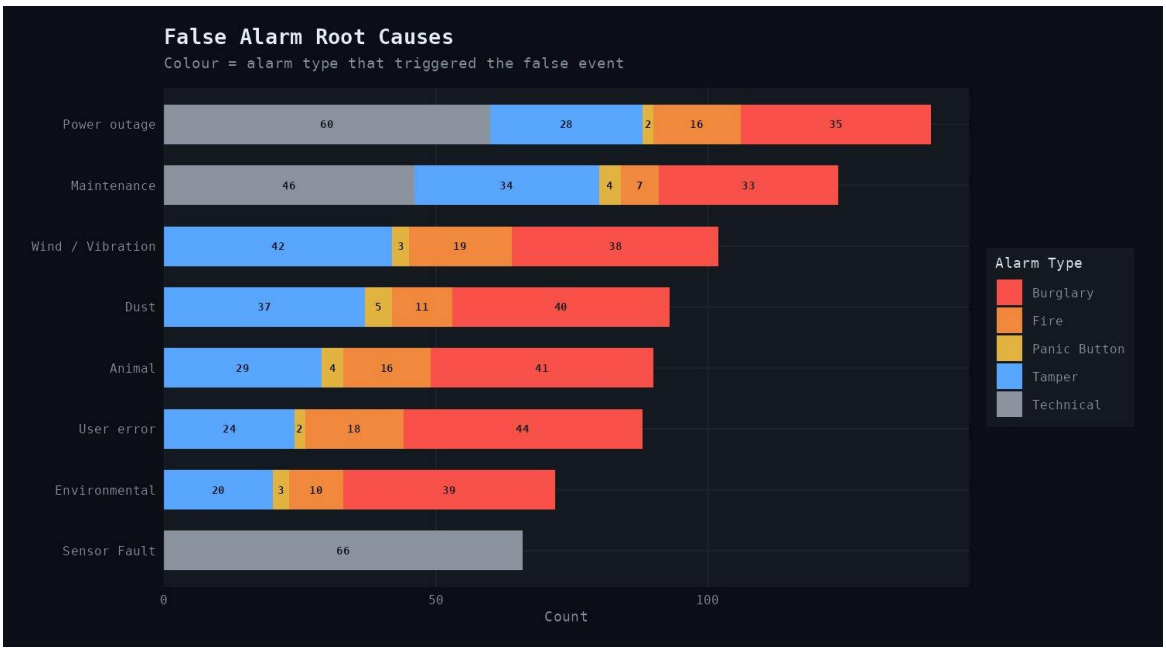
E08 is the only agent flagged above the mean + 1 SD threshold, with a false alarm rate of 86.1% — more than 10 percentage points above the next highest agent (E07 at 75.6%). The fleet mean (dashed amber line) sits at approximately 68%, with the alert threshold (dotted red line) at approximately 78%. The broad range across agents (58.9% for E01 to 86.1% for E08) suggests both individual performance variation and a structurally high fleet-level false alarm rate that warrants operational review.



Plot 3. False alarm rate by agent, ranked. Dashed line = fleet mean; dotted line = mean + 1 SD alert threshold. E08 is the only agent above the threshold.

Plot 4 — False Alarm Root Causes

Power outage and Maintenance are the two highest-volume root causes (~141 and ~124 false alarms respectively), driven primarily by Technical (grey) and Burglary (red) alarm types. Sensor Fault is notable for being almost entirely a single-type issue (Technical alarms only, 66 events), suggesting a systematic equipment reliability problem rather than environmental or human factors. User error (44 Burglary, 24 Tamper, 18 Fire, 2 Panic) indicates training opportunities across multiple alarm categories.



Plot 4. False alarm counts by root cause, stacked by the alarm type that triggered the false event. Ordered by total false alarm count per root cause.

Plot 5 — Monthly Signal Volume and Rate Trends

Total signal volume is relatively stable across the six months (range: 186 in February to 213 in June), with no clear upward or downward trend. The incident rate (red line, bottom panel) fluctuates between 26.3% and 29.6% with no directional trend. The false alarm rate (amber line) is similarly stable at 67.7%–71.3%. This stability is a key finding: the operational workload and quality metrics are not systematically improving or deteriorating at the fleet level, meaning any detected agent-level patterns represent individual rather than systemic issues.

Plot 5. Monthly signal volume (top: blue=total, red=incidents) and rate trends (bottom: incident rate and false alarm rate). Jan–Jun 2025.

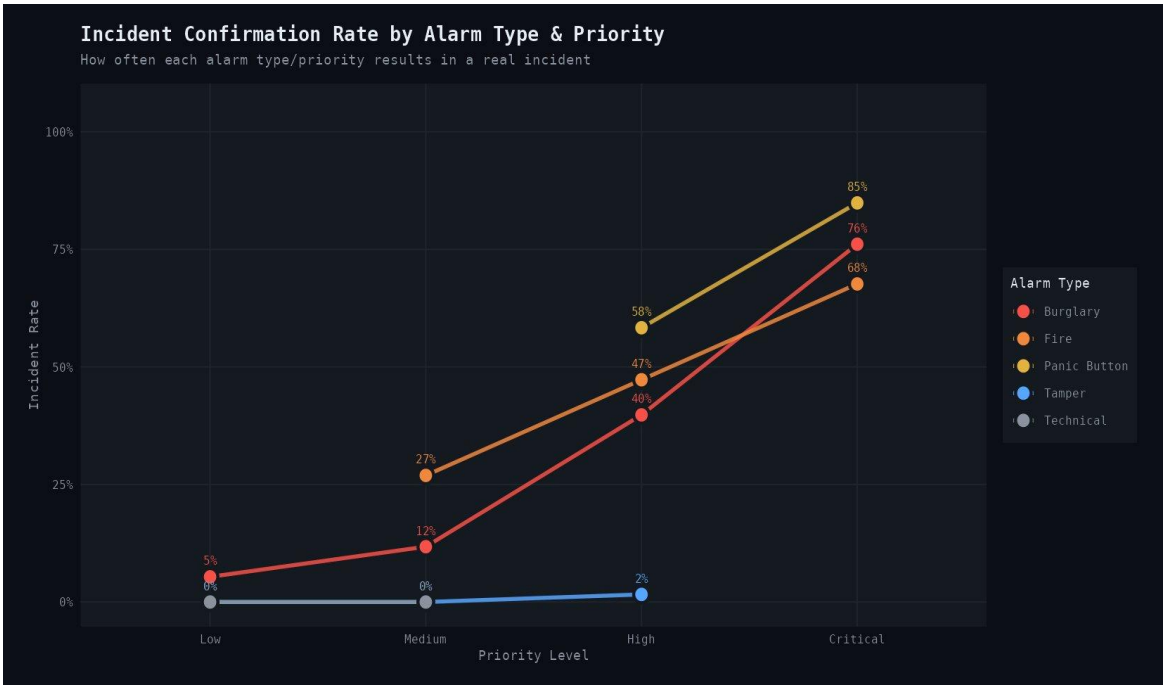
Plot 6 — Agent × Shift Response Time Heatmap

The heatmap confirms that response time differences are primarily agent-driven rather than shift-driven. Senior agents (E01, E02) show green cells across all three shifts (2.1–2.8 minutes), while E08's entire row is uniformly red (14.2–14.7 minutes). There is no systematic pattern where any shift is consistently slower across all agents, which rules out shift-level explanations (e.g. night-shift fatigue) as the primary driver of performance variation.

Plot 6. Average response time heatmap for each agent-shift combination. Green = fast; red = slow. Cell labels show mean response time and signal count.

Plot 7 — Incident Confirmation Rate by Alarm Type and Priority

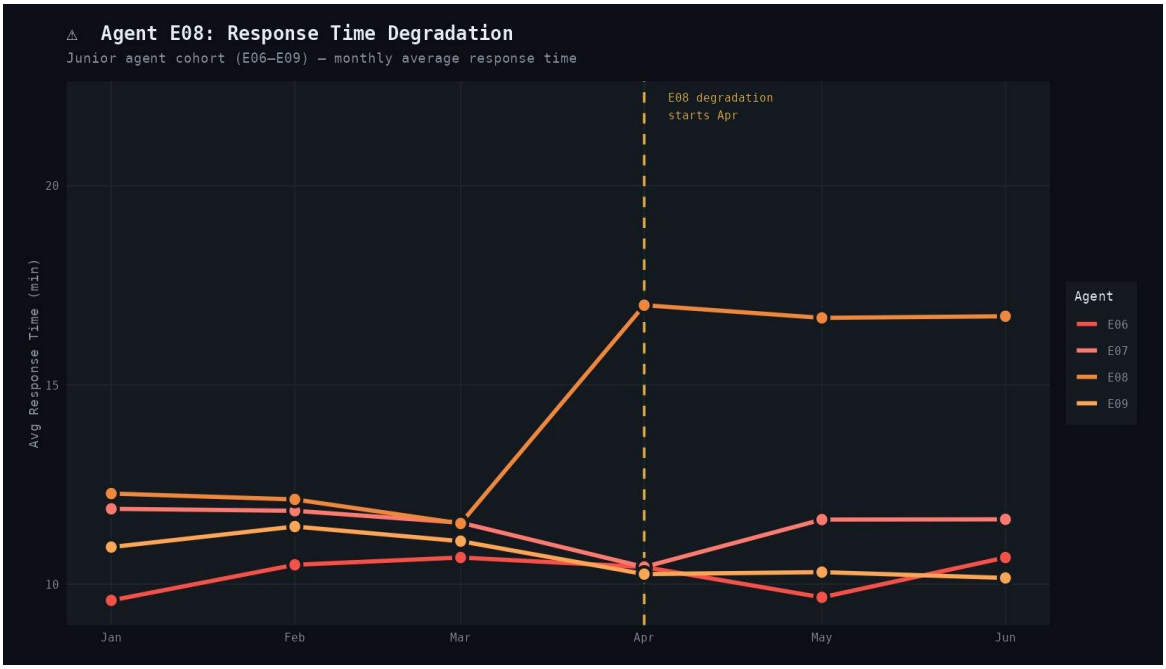
Panic Button achieves the highest Critical-priority confirmation rate at 85%, followed by Fire (76%) and Burglary (68%). The steep positive slope for all three active alarm types demonstrates that the priority classification system is working correctly — higher priority genuinely predicts a higher real-incident rate. Tamper and Technical alarms remain near-zero across all priority levels, confirming that these alarm types are almost structurally non-incident regardless of how they are classified, which raises the question of whether current triage protocols appropriately differentiate response protocols for these types.



Plot 7. Incident confirmation rate by alarm type across priority levels. Lines connect priority tiers per alarm type. Panic Button and Fire show the steepest priority-to-confirmation slopes.

Plot 8 — Agent E08: Response Time Degradation

The degradation pattern for E08 is unmistakable. From January through March, E08's average monthly response time is 11–12 minutes — consistent with peers E06 and E07. Starting in April, E08's response time jumps to 16.4 minutes and remains elevated (15.4–15.7 minutes) through May and June, while peers E06, E07, and E09 remain stable or improve slightly. The formal t-test (Step 15h) confirms this before/after difference is statistically significant. This type of individual-level time-series monitoring is a key use case for SOC analytics — it can surface burnout, training gaps, or workload imbalances before they become operational risks.



Plot 8. Monthly average response time for the Junior agent cohort. E08 diverges sharply from peers after April 2025 (dashed reference line).

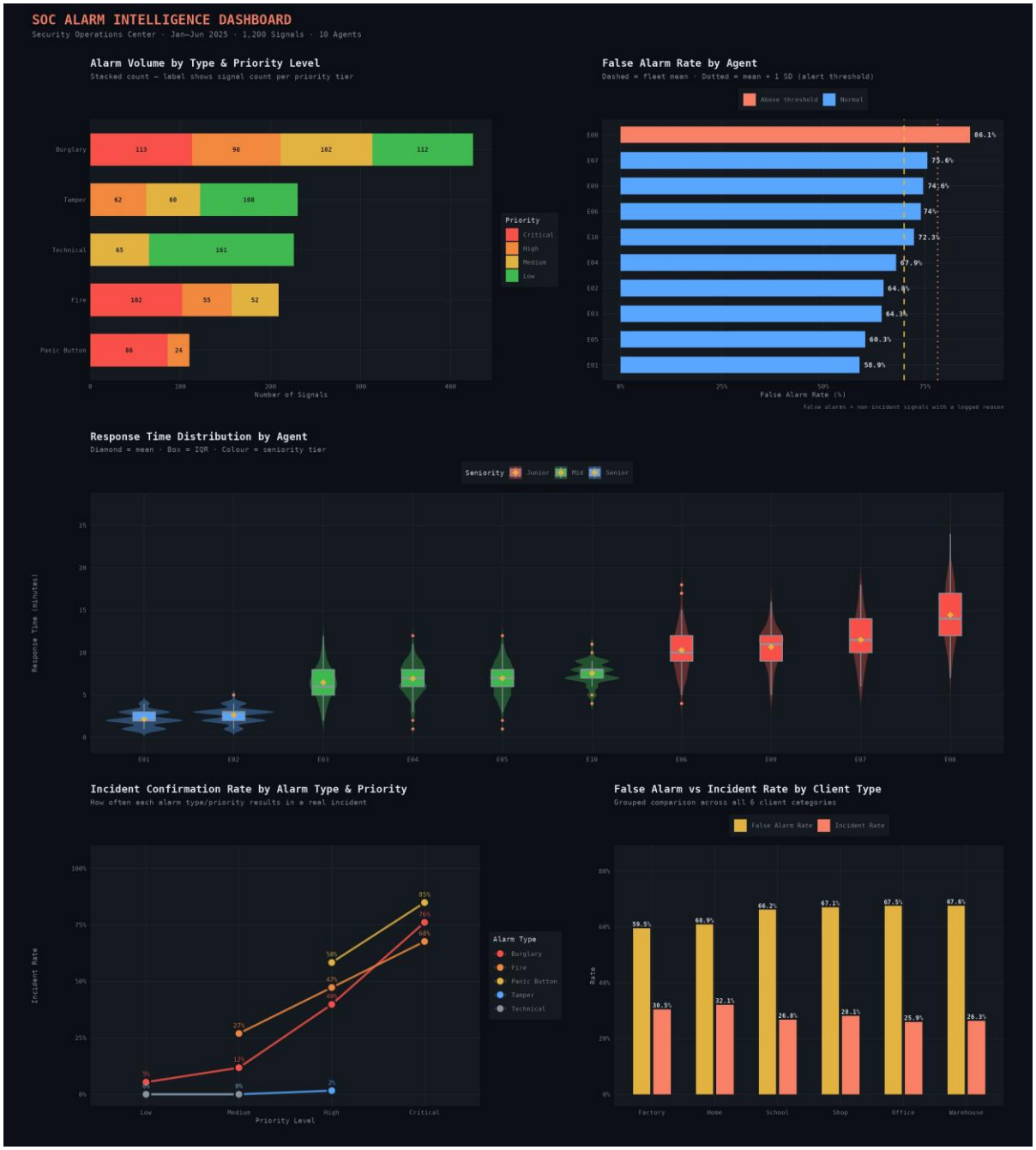
Plot 9 — False Alarm vs Incident Rate by Client Type

Factory and Home client types have the lowest false alarm rates (59.5% and 60.9%) and the highest incident rates (30.5% and 32.1%), suggesting these environments generate more genuine security events relative to false activations. School, Shop, Office, and Warehouse all cluster around 66–68% false alarm rates with incident rates of 25–28%. This pattern could reflect differences in alarm system quality, occupancy patterns, or staff training at client sites. The relatively narrow range across client types (59–68% false alarm) suggests client type is a less powerful predictor of false alarm rate than agent identity.

Plot 9. False alarm rate (amber) and incident rate (red) for each of the six client types. Factory and Home show the highest incident-to-false-alarm ratios.

Plot 10 — Composite SOC Dashboard

The composite dashboard brings together five of the nine plots into a single A3-scale briefing view, built with patchwork. The layout prioritises the two most actionable outputs (alarm volume and false alarm rate by agent) in the top row, the full agent response time distributions in the middle, and the two strategic pattern plots (incident confirmation rate and client type breakdown) at the bottom. The dark theme and monospaced title font are intentional design choices to match the visual language of operational SOC environments.



Plot 10. Composite SOC Alarm Intelligence Dashboard. Five key plots composed into a single briefing layout using patchwork. Generated at 18×20 inches, 150 dpi.

References and Technical Notes

R packages: readxl, dplyr, tidyr, lubridate, stringr, forcats, ggplot2, scales, patchwork, viridis, broom, knitr.

Statistical methods: one-way ANOVA, Kruskal-Wallis test, chi-squared test of independence, Spearman rank correlation, logistic regression (binomial, logit link), Welch two-sample t-test, McFadden pseudo R-squared.

E08 performance degradation was deliberately seeded into the synthetic dataset as a test case for the automated flagging and time-series monitoring components of the script.

All datasets are fully synthetic. Code available at: github.com/[your-username]